

STAT 418 · FINAL PROJECT · JUNE 2026

Predicting NBA Playoff Underperformance

A game-level machine-learning system with live API and web app

PRESENTED BY

Hanzhang Yuan

DATE

June 4, 2026

Sports Analytics × Machine Learning

Why Some Stars Fade in the Playoffs

THE QUESTION

Given only data available before tip-off, will a given NBA player underperform their regular-season Game Score baseline in this specific playoff game?

WHY IT MATTERS

Tighter defenses • Targeted scouting • Shorter rotations

Some stars rise; others fade. Quantifying fade-risk before a series starts has real value for fans, analysts, and front offices.

PROBLEM TYPE

Binary

Underperform yes / no

UNIT OF ANALYSIS

Player-game

Not player-season

DATA RANGE

22 seasons

2003-04 to 2024-25

Scraping 22 Seasons of NBA Box Scores

DATA SOURCE

NBA Stats API via the `nba_api` Python wrapper

Player game logs and team advanced stats for 2003-04 through 2024-25

SCRAPING PIPELINE

- | | |
|------------------------------|-----------------------------|
| 1. <code>nba_api</code> | rate-limited 1.2 s/req |
| 2. Raw JSON cache | data/raw/, resumable |
| 3. <code>parse_raw.py</code> | → Parquet, Game Score |
| 4. <code>league_norms</code> | per-season μ , σ |

API REQUESTS

~25,000
across 22 seasons

SCRAPE TIME

~7 hours
one-time, overnight

PLAYER-GAMES

~280,000
regular + playoff

MODELING SET

~45,000
playoff-only

Three Models, 45 Features, Strict Time Split

MODELS COMPARED

BASELINE

Logistic Regression

Interpretable; sets floor

MID-TIER

Random Forest

Non-linear interactions

PRIMARY

XGBoost

SOTA on tabular data; Optuna-tuned 50 trials

EVALUATION

Primary: ROC-AUC (threshold-free)

Secondary: Accuracy • Precision • Recall • F1

Sliced: by series_game_number and player tier

STRICT TIME-ORDERED SPLIT

TRAIN

2003-21 (18 seasons)

VAL

2021-23 (2 sns)

TEST

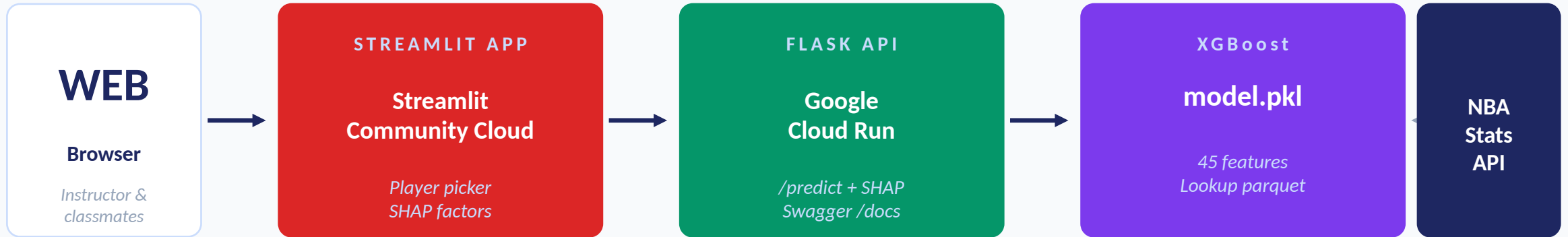
2023-25 (2 sns)

Important: *TimeSeriesSplit CV, never random k-fold*

Information lock at tip-off: every feature is verified against future-data leakage by 7 hard pytest assertions in tests/test_no_leakage.py

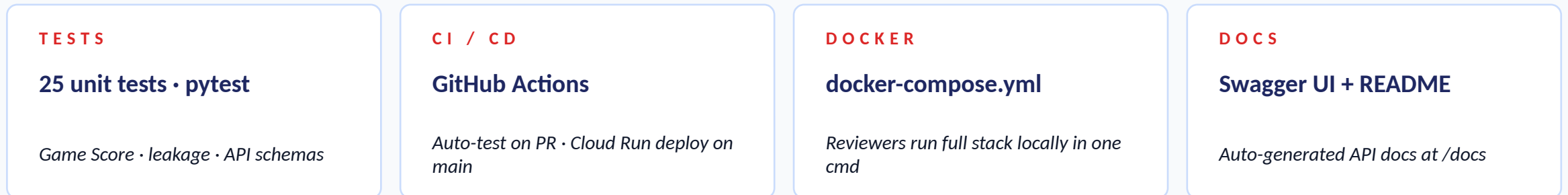
Deployed System: API + App + Cloud

USER



trained on

ENGINEERING QUALITY



Walkthrough: Select a Player → Get Risk + Explanation

SIMPLE MODE — 5 INPUTS

PLAYER

Luka Dončić

OPPONENT

Boston Celtics

SEASON

2024-25

SERIES GAME #

5

LOCATION

Away

SERIES SCORE

2-2

RESULT — POWERED BY SHAP

UNDERPERFORM PROBABILITY

67%

HIGH RISK

TOP FACTORS

UP

Opponent defense is top of the league

+12%

UP

Historically weak in Game 5s

+8%

UP

Away game adds risk

+5%

DN

Recent series momentum is positive

-4%

Challenges, Lessons, and What's Next

BIGGEST CHALLENGES

Hidden leakage in expanding-window features

Solved by writing 7 pytest assertions BEFORE the pipeline; caught 2 real bugs

nba_api rate limiting

Solved by 1.2 s/req throttle + exponential backoff + disk cache (resumable)

Era differences (2003 vs 2024)

Solved by per-season z-score normalization — one-line transformation

Heterogeneity across players

Solved by encoding personal history (past_game1, past_game7, past_elim) as features

LESSONS LEARNED

- > Always express data-flow constraints (no leakage) as hard tests, not as comments
- > Cache aggressively in long-running jobs — resumability turns 8-hour scrapes from terrifying to routine
- > Per-row SHAP explanations make a binary classifier feel like a tool, not a black box
- > Streamlit Community Cloud + Cloud Run is the fastest path from notebook → deployed service

Next: scheduled weekly scrape • injury features • direct GS regression • 2025-26 in-progress predictions